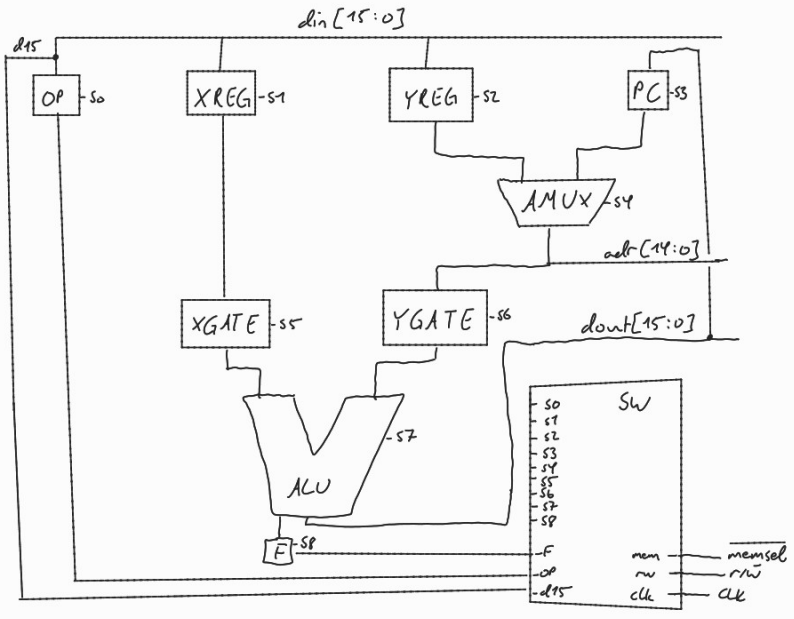
Kevin Küster

Zusammenfassung: Rechnerorganisation

Rechnerorganisation

MPRO7 (Minimal Prozessor)

Aufbau



Steuerung

OP: 0 ≙ add, 1 ≙ nor

XGate: 0 ≙ '1', 1 ≙ XREG

YGATE: 0 ≙ '0', 1 ≙ YREG

AMUX: 0 ≙ YREG, 0 ≙ PC

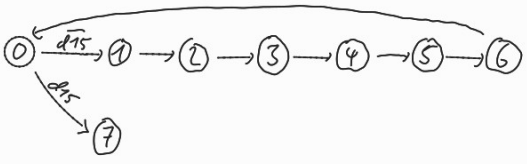
Adressbereiche

$0000 - $3fff : ROM

$4000 - $7dff : RAM

$7e00 - $FFFF : I/O - Hardware

Zustandsdiagramm



0.) 1. Befehlswort laden, PCH

1.) 1. Operanden laden

2.) 2. Befehlswort laden

3.) 2. Operanden laden

4.) Operation durchführen

5.) 2. Befehlswort laden, PCH

6.) Ergebnis speichern

q	r/w̄	memsel̄	[s] 0 1 2 3 4 5 6 7 8	Funktion
0	1	0	x 1 1 1 1 0 1 0 0	1.BFW, PCH
1	1	0	x 1 x 0 0 x x x x	1.OP
2	1	0	1 0 1 0 1 x x x x	2.BFW
3	1	0	0 0 1 0 0 x x x x	2.OP
4	0	1	x 1 x 0 0 1 1 OP 1	OP
5	1	0	x 0 1 1 1 x x x 0	2.BFW, PCH
6	0	0	x x x 0 0 1 0 0 0	Erg speichern
7	x	1	x x x F̄ x 1 0 0 1	br

Befehle

add, nor, br (Branch; Sprung)

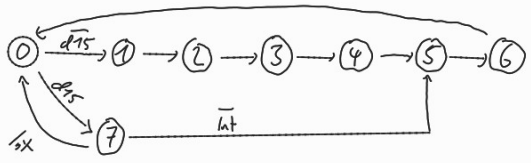
Befehl	Speicher	Unterstützte Flags
br adr	15 0 / 0 adr	F=0: br adr F=1: F=0, PCH
add adr1 adr2 ↳ adr2 = adr1 + adr2	15 0 / 0 adr 0 adr	Carry-Flag
nor adr1 adr2 ↳ adr2 = adr1 nor adr2	15 0 / 0 adr 1 adr	zero-Flag

Steuerwerk

Hartverdrahtet	Mikroprogrammiert
- Steuerung direkt durch Logik-Gatter oder FlipFlops	- Steuerung durch Mikro-Clock
- Keine Nachträgliche Änderung	- Veränderbar
- Sehr schnell	- Langsamer
- Weniger Hardware-Komponenten	- Aufwändiger (Hardware)
- Nicht Flexibel	- Flexibel
	- Eine instruktion pro Steuersit

Interrupt

Interrupt - Hartverdrahtet



$\overline{Int} = \overline{F} \cdot \overline{IRQ} \cdot \overline{IE} \;\; 1\,Q7$ (F: Flag, IRQ: Interrupt-Request, IE: Interrupt-Enable) } Voraussetzungen

Bei Int (=1) wird Interrupt verworfen und Befehl normal ausgeführt. Bei $\overline{Int}$ wird Interrupt durchgeführt:

Aktueller Befehl wird gespeichert unter der ersten Adresse (hier RAM: $4000):

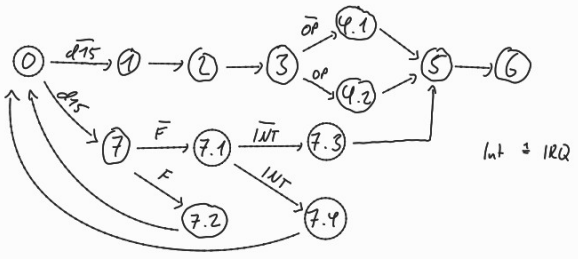
- Akt. Befehlswort wird geladen, PC=1 (via. Konstantenerzeugung) → Akt. Befehlswort speichern (Q7 → Q5)
- Durch schreiben auf Adresse 1 wird IE=0 gesetzt → Während Interrupt kein weiterer Interrupt möglich
- ISR (Interrupt-Service-Routine) wird durchgeführt
- Nach Ablauf ISR: Akt. Befehl (Adr1) wird geladen → IE=1

Interrupt - Mikro

⇒ Mikroprogrammierbares Steuergerät mit Interrupt-Unterstützung

↳ (Q4) aufteilen (Q4.1) - add ($\overline{OP}$), (Q4.2) - nor (OP)

↳ (Q7) aufteilen auf Basis der Voraussetzungen von $\overline{Int}$: F und IRQ (IE ist Steuerungsflag, KEINE Zustandsflag)



Int ≙ IRQ

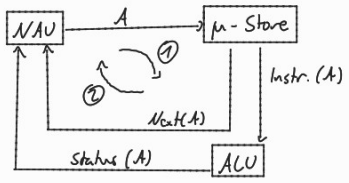
Interrupt vs Polling

Interrup wird durch Zustandsveränderung eingeleitet ⇒ Aktion, komplexer

Polling prüft dauerhaft auf Veränderung ⇒ Reaktion, ggf. unzuverlässig aufgrund von delay, Dauerhafte Belastung der CPU

Pipelining - Arten - Seriell

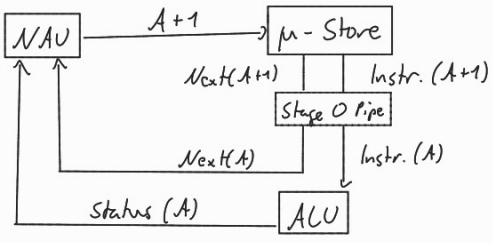
Grundsaustein



- NAU: Next Address Unit
- µ-Store: Mikrospeicher
- ALU: Arithmetic & logic Unit

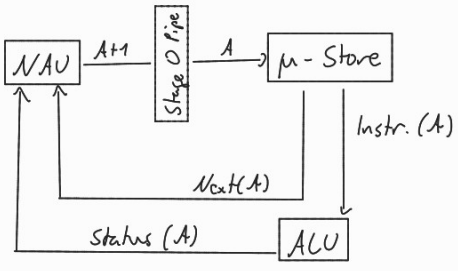
- Instruction (A): Steuerbits der zeitige
- Next(A): Folge-Informationen
- A: Adresse
- A+1: Folge-Adresse
- Status (A): Status-Flag (s) [Zero, Negative, Overflow, Carry]
- ① ALU erzeugt Status: Inst. (A) $\xrightarrow{ALU}$ Status (A)
- ② NAU „berechnet" A+1: Next(A) & Status (A) $\xrightarrow{NAU}$ A+1

8.4 Instruction Based Design



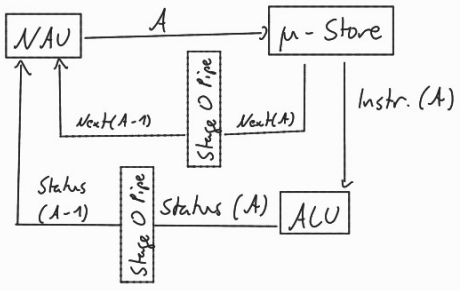
- Einfache Programmierung
- Hohe Ausführungszeit

8.5 Address-Based-Design



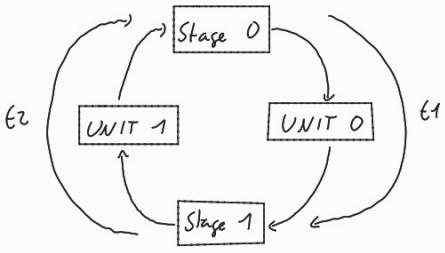
- Einfache Programmierung
- Hohe Ausführungszeit

8.6 Data-Based-Design



- Schwere Programmierung
- Hohe Ausführungszeit

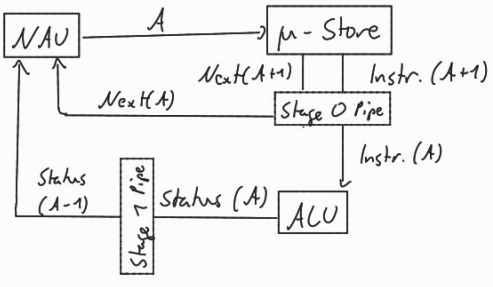
Parallele Verarbeitung



$t = \max(t1, t2)$

⇒ Kombination der einzelnen Designs

Instruction-Data-Based



- ~2x Geschw.
- schwere Programmierung

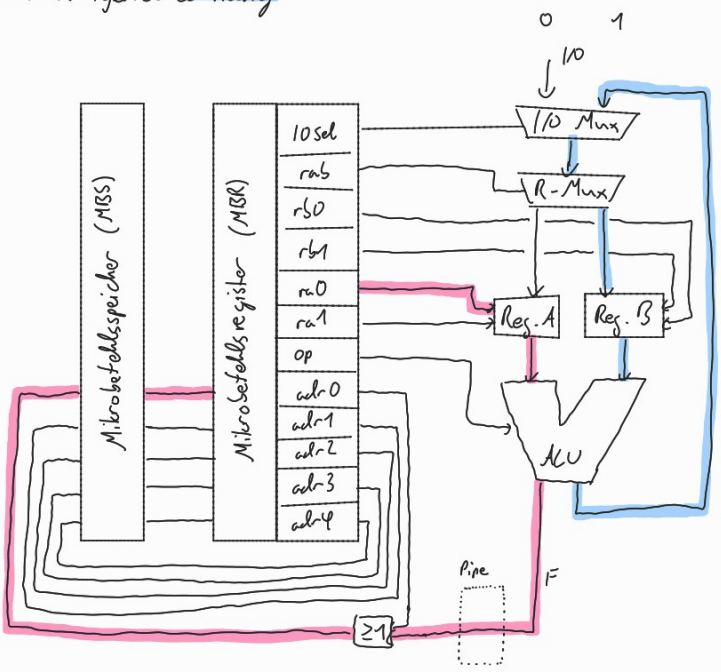
Instruction-Addressed Based

- ~2x Geschw.
- NAU problematisch; Next(A) & Status (A) $\xrightarrow{NAU}$ A+2

Address-Data-Based

- schnell
- Adress-Generierung erschwert: Next(A) & Status (A-1) $\xrightarrow{NAU}$ A+1

Pfadlängenberechnung



Millisekunde (ms)	$1/1.000\,s$
Mikrosekunde (µs)	$1/1.000.000\,s$
Nanosekunde (ns)	$1/1.000.000.000\,s$
Picosekunde (ps)	$1/1.000\,ns$

$ns \rightarrow MHz: \quad x\,ns = x \cdot 10^{-9}\,s = \frac{1}{x \cdot 10^{-9}}\,Hz = \frac{1}{\frac{x}{10^9}}\,Hz = \frac{10^9}{x}\,Hz$

$= \frac{10^3}{x} \cdot 10^6\,Hz = \frac{1000}{x}\,MHz$

$MHz \rightarrow ns: \quad x\,MHz = x \cdot 10^6\,Hz = \frac{1}{x \cdot 10^6}\,s = \frac{10^9}{x \cdot 10^6}\,ns$

$= \frac{1000}{x}\,ns$

- Adresspfad : RegA (lesen) ↦ ALU ↦ Gatter ↦ MBS ↦ MBR (Setup)
- Berechnungspfad : RegB (lesen) ↦ ALU ↦ I/O Mux ↦ R Mux ↦ Reg. B (Setup)

Effizienter via Pipelining

Inst.-Data-Based Pipelining (Parallel)

Auswirkungen: - Taktverzögerung (Status-Flag einen Takt später)
- Mehr Programmieraufwand
- Parallele Verarbeitung ⇒ schneller

Pfad A: Reg. A (lesen) → ALU → Pipe (Setup)

Pfad B: Pipe (lesen) → Gatter → MBS → MBR (Setup)

⇒ Maximale Betriebsfreq.: Langsamster Datenpfad

Optimierungsmöglichkeiten

1. Bessere Hardware (schnellere Durchlaufzeit)
2. ALU mit mehr unterstützen Operationen

Mikroprogrammierung

Akt. Adr.	adr4	adr3	adr2	adr1	adr0	OP	ra1	ra0	rb1	rb0	rab	iosel	Beschreibung
0	0	0	0	0	1	0	0	0	0	0	0	0	A0 ← I/O
1	0	0	0	1	1	0	0	0	0	1	1	0	B1 ← I/O
3	0	1	0	0	0	0	0	0	0	1	0	1	A0 ← add B1 A0
4	0	1	0	0	1	1	0	0	0	1	0	1	A0 ← nor B1 A0
5	0	0	0	0	1	0	0	0	0	0	0	0	A0 ← I/O

Hier ≥1 — adr0, F } 1 ⇒ Keine Fallunterscheidung ≙ true

0 ⇒ Fallunterscheidung F → xxxx1 ; $\bar{F}$ → xxxx0

Endlosschleife: Input A0 einlesen; Input B1 einlesen; add B1 A0 — F → Schleife ; $\bar{F}$ → nor B1 A0; Schleife

Assembly

Gruppen: Arithmetisch, Logisch, Sprung, Vergleich

Lable (Zwischenvariable): @NAME dc.w $xxxx
(dc.w = declear word, $xxxx = Hex-Wert)

Löschen:
```
@FFFF dc.w $FFFF
nor @FFFF adr
```

Move adr1 → adr2:
```
@FFFF dc.w $FFFF
nor @FFFF adr2
add adr1 adr2
```

Invert: `nor adr adr` $\quad \bar{a} = \overline{a+a}$

Or:
```
nor adr1 adr2
nor adr2 adr2
```
$a+b = \overline{\overline{a+b}}$

AND:
```
nor adr1 adr1
nor adr2 adr2
nor adr1 adr2
```
$a \cdot b = \overline{\bar{a}+\bar{b}}$

NAND:
```
nor adr1 adr1
nor adr2 adr2
nor adr1 adr2
nor adr2 adr2
```

Neg:
```
@0001 dc.w $0001
nor adr adr
add @0001 adr
```
$-a = \bar{a} + 1$

Inc:
```
@0001 dc.w $0001
add @0001 adr
```

Dec:
```
@FFFF dc.w $FFFF   → $FFFF ≙ -1
add @FFFF adr
```

Sub: adr1 = adr1 − adr2
```
@0001 dc.w $0001
nor adr2 adr2
add @0001 adr2
add adr2 adr1
```

Mult: adr3 = adr1 · adr2
```
@FFFF dc.w $FFFF
nor @FFFF adr3
nor @FFFF adr4
@0001 dc.w $0001
add adr2 adr4
nor adr4 adr4
add @0001 adr4      adr4 = -a adr2
loop:
    nor adr4 adr4 }
    nor adr4 adr4 } adr4 = 0 ?
    br end        } ↳ end
    add adr1 adr3
    add @0001 adr4  → adr4 = adr4 - 1
    br loop
end:
```

Left-Shift: 0101 → 1010 ≙ ·2
1010 → 1|0100
```
add adr adr
```
(SHL (Shift Logical Left) ≙ SAR (Shift Arithmetik Left))

L-Rotate: 1010 → 0101
```
@0001 dc.w $0001
add adr adr
br end
add @0001 adr
end:
```

SHR:
(5) 0101 → (2) 0010|1
(-5) 1011 → (5) 0101|1

SAR:
(5) 0101 → (2) 0010|1
(-5) 1011 → (-3) 1101|1

{pre}: adr1 = adr1 − adr2
```
@0001 dc.w $0001
nor adr2 adr2
add @0001
add adr2 adr1
```

JE (Jump Equal): adr1 = adr2
```
{pre}
nor adr1 adr1
nor adr1 adr1
br false
br true
```

JG (Jump Greater): adr1 > adr2
```
{pre}
add adr1 adr1
br false
br true
```

JL (Jump Less): adr1 < adr2
```
{pre}
add adr1 adr1
br true
br false
```

%2:
```
NOR adr
br true
br false
```

Definitionen - Begriffe

Barrel - Shifter

Verschiebelogik: In einem Takt kann eine beliebige Anzahl von Bits um n stellen verschoben werden

Aufbau durch Schalter und Multiplexer, die alle Bits miteinander verbinden.

Beispiel: Eingabe: ABCD. Ausgaben: ABCD, BCDA, CDAB, DABC

↳ 4x Dateneingang, 2x Steuereingang, 4x Datenausgang

↳ Zuweisung der Ausgabe via Steuerbit-Eingaben

Ripple - Carry - Addierer

Mehrere in Reihe geschaltete Volladdierer; Berechnung sequentiell und erst abgeschlossen, wenn Übertrag an höchstwertigster Stelle angekommen ist

↳ „durchgerippelt"

- geringe Komplexität
- Langsam

Carry - look - Ahead - Addierer

Aufbau analog zum RCA, aber ergänzt durch Logikgatter, um Überträge parallel zu berechnen

- höhere Komplexität (Hardware)
- sehr schnell

MPROZ: RCA

One - Hot - Steuerung

1 - aus - n - Steuerung: Zustandssteuerung bei der jeder Zustand durch einen FlipFlop realisiert wird.

Während des Prozesses kann nur einer dieser FFs gleichzeitig aktiv ('1') sein („Hot").

Reset entspricht einer logischen '0':

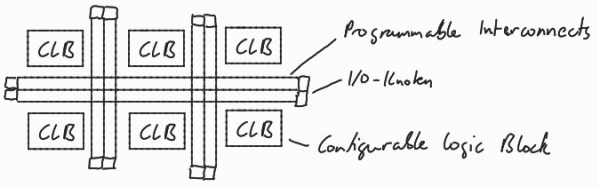
1.) Asynchroner Reset: Sofortiges Zurücksetzen aller FFs via Signal, unabhängig von Takt

2.) Synchroner Reset: Reset fungiert als Flag und wird mit nächstem Takt gültig ⇒ Abstimmung mit CLK

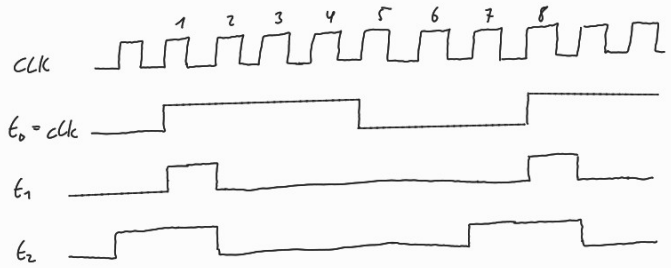
FPGA (Field Programmable Gate Array)

Das FPGA ist eine elektronische Schaltung, in die eine beliebige logische Schaltung geladen (programmiert) werden kann.

Es besteht aus einer Anordnung von veränderbaren Logik-Blöcken (Configurable Logic Block (CLB)), Routing Ressourcen (Ansteuerbare Verkabelung) und I/O-Knoten als Schnittstelle. CLB können z.B. über eine lookUpTable betrieben werden.



Clock



t_0: 8 Takte (↑↓ ... ↑), CLK: 32 MHz → $t_0 = \frac{32\,MHz}{8} = 4\,MHz$

t_1: I/O → Input/Output; RAM/ROM je nach Adressbereich

t_2: r/w → read/$\overline{write}$